

❑ Object-Oriented Programming — Complete Interview Q&A Guide

Covers every OOP concept from **Basic** → **Intermediate** → **Advanced**, with detailed answers, examples, and edge cases.

❑ Table of Contents

1. [Core Concepts & Fundamentals](#)
 2. [Classes & Objects](#)
 3. [Encapsulation](#)
 4. [Inheritance](#)
 5. [Polymorphism](#)
 6. [Abstraction](#)
 7. [Constructors & Destructors](#)
 8. [Interfaces & Abstract Classes](#)
 9. [Access Modifiers](#)
 10. [Static Members](#)
 11. [Method Overloading & Overriding](#)
 12. [Association, Aggregation & Composition](#)
 13. [Coupling & Cohesion](#)
 14. [SOLID Principles](#)
 15. [Design Patterns](#)
 16. [Exception Handling in OOP](#)
 17. [Generics & Templates](#)
 18. [Memory Management & Object Lifecycle](#)
 19. [Inner Classes & Anonymous Classes](#)
 20. [Advanced OOP Concepts](#)
-

1. Core Concepts & Fundamentals

Q1. What is Object-Oriented Programming (OOP)?

A: OOP is a programming paradigm that organizes software design around **data (objects)** rather

than functions and logic. An object is a self-contained unit that bundles both **state (attributes/fields)** and **behavior (methods/functions)**.

The four core pillars of OOP are:

- **Encapsulation** — hiding internal state and requiring all interaction through methods.
- **Inheritance** — deriving new classes from existing ones.
- **Polymorphism** — one interface, multiple implementations.
- **Abstraction** — exposing only essential details, hiding complexity.

Q2. What are the advantages of OOP over procedural programming?

A:

Aspect	Procedural	OOP
Data handling	Data and functions are separate	Bundled in objects
Reusability	Limited	High via inheritance
Maintainability	Harder at scale	Easier with modularity
Security	Low (global data)	High (encapsulation)
Real-world modeling	Poor	Natural fit

Additional benefits: easier debugging, code reusability, scalability, and team collaboration.

Q3. What is the difference between a class and an object?

A:

- **Class** is a **blueprint or template** that defines properties and behaviors. It does not occupy memory by itself.
- **Object** is an **instance of a class** — a concrete entity created from the blueprint that occupies memory.

```
// Class (blueprint)
class Car {
    String color;
    void drive() { System.out.println("Driving..."); }
}

// Object (instance)
Car myCar = new Car();    // myCar is an object
myCar.color = "Red";
myCar.drive();
```

Q4. What is an instance variable vs a class variable?

A:

- **Instance variable:** Declared inside a class but outside methods. Each object has its own copy.
- **Class variable (static):** Shared across all objects of the class. Only one copy exists regardless of how many objects are created.

```
class Counter {  
    int instanceCount = 0;        // instance variable – unique per object  
    static int totalCount = 0;    // class variable – shared by all  
}
```

Q5. What is the difference between procedural and object-oriented programming?

A:

- **Procedural:** Program is a sequence of instructions. Functions operate on data passed to them. Example: C.
- **OOP:** Program is a collection of interacting objects. Data and behavior are co-located. Example: Java, Python, C++.

Key shift: In OOP, you think "what objects exist and how do they collaborate?" rather than "what steps does the program take?"

2. Classes & Objects

Q6. What is a class in OOP?

A: A class is a user-defined data type that serves as a blueprint for creating objects. It defines:

- **Attributes** (data members / fields)
- **Methods** (member functions / behavior)
- **Constructors** (initialization logic)
- **Access control** (public/private/protected)

```
class BankAccount:  
    def __init__(self, owner, balance):  
        self.owner = owner  
        self.__balance = balance    # private  
  
    def deposit(self, amount):  
        self.__balance += amount  
  
    def get_balance(self):  
        return self.__balance
```

Q7. What is an object? How is it created?

A: An object is a runtime instance of a class that holds actual data. Creation involves:

1. **Memory allocation** (heap in most languages)

2. **Constructor invocation** (initialization)

```
// Java
Dog d = new Dog("Buddy", 3);

// Python
d = Dog("Buddy", 3)

// C++
Dog* d = new Dog("Buddy", 3); // heap
Dog d("Buddy", 3);           // stack
```

Q8. What is **this** keyword?

A: **this** is a reference to the **current object** inside a class method or constructor. It is used to:

- Distinguish instance variables from local variables with the same name.
- Call another constructor in the same class (constructor chaining).
- Pass the current object as an argument.

```
class Person {
    String name;
    Person(String name) {
        this.name = name; // 'this.name' = instance var, 'name' = parameter
    }
}
```

Q9. What are the differences between stack and heap memory in context of objects?

A:

- **Stack:** Stores primitive variables and object references. LIFO, auto-managed. Fast but limited.
- **Heap:** Stores actual objects. Managed by garbage collector (Java/Python) or manually (C+). Slower but large.

```
int x = 5;           // x stored on stack
Dog d = new Dog();   // reference 'd' on stack, Dog object on heap
```

Q10. What is an anonymous object?

A: An object created without assigning it to a reference variable. Used when the object is needed only once.

```
new Dog("Rex").bark(); // Anonymous object – no variable holds the reference
```

3. Encapsulation

Q11. What is Encapsulation?

A: Encapsulation is the OOP principle of **bundling data (fields) and methods that operate on that data into a single unit (class)**, and **restricting direct access** to some components using access modifiers.

Benefits:

- Data hiding and security
- Controlled access via getters/setters
- Flexibility to change internals without breaking external code

```
class Employee {  
    private double salary;    // hidden  
  
    public double getSalary() { return salary; }  
    public void setSalary(double s) {  
        if (s > 0) salary = s;    // validation logic  
    }  
}
```

Q12. What is the difference between encapsulation and data hiding?

A:

- **Encapsulation** = bundling data + methods together (broader concept).
- **Data hiding** = preventing direct access to internal state (subset of encapsulation using access modifiers).

All data hiding is encapsulation, but encapsulation also includes binding behavior to data.

Q13. What are getters and setters? Why use them instead of public fields?

A: Getters and setters are public methods that provide controlled access to private fields.

Advantages over public fields:

- **Validation:** Can enforce constraints before setting a value.
 - **Read-only / Write-only:** Can expose only getter (read-only field).
 - **Computed properties:** Getter can return derived values.
 - **Backward compatibility:** Internals can change without affecting consumers.
-

Q14. Can a class be fully encapsulated with no getters or setters?

A: Yes. If a class operates entirely internally (e.g., a helper class that exposes only high-level methods), getters/setters are unnecessary. Example: a `PasswordHasher` class that accepts a

string and returns a hash — the internal algorithm is fully hidden.

4. Inheritance

Q15. What is Inheritance?

A: Inheritance is the mechanism by which a **child class (subclass/derived)** acquires properties and behaviors of a **parent class (superclass/base)**, promoting code reuse.

```
class Animal {
    void breathe() { System.out.println("Breathing..."); }
}

class Dog extends Animal {
    void bark() { System.out.println("Woof!"); }
}

Dog d = new Dog();
d.breathe(); // Inherited from Animal
d.bark();    // Own method
```

Q16. What are the types of inheritance?

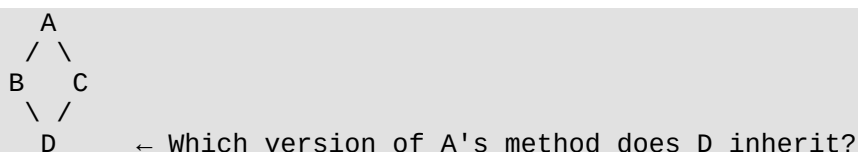
A:

Type	Description	Supported in Java?
Single	One child, one parent	
Multilevel	A → B → C (chain)	
Hierarchical	One parent, multiple children	
Multiple	One child, multiple parents	(use interfaces)
Hybrid	Combination of above	(use interfaces)

Java avoids multiple class inheritance to eliminate the **Diamond Problem**.

Q17. What is the Diamond Problem?

A: When a class inherits from two classes that both inherit from a common base class, ambiguity arises about which version of an inherited method to use.



Java's solution: Classes can only have one parent. Multiple inheritance is allowed through interfaces (which have default methods with explicit resolution rules).

C++ solution: Virtual inheritance.

Q18. What is the **super** keyword?

A: **super** refers to the **parent class**. Used to:

- Call parent class constructor: `super(args)`
- Call parent class method: `super.methodName()`
- Access parent class fields (if not hidden)

```
class Vehicle {
    Vehicle(String type) { System.out.println("Vehicle: " + type); }
    void show() { System.out.println("Vehicle"); }
}

class Car extends Vehicle {
    Car() {
        super("Car");    // calls Vehicle constructor
    }
    void show() {
        super.show();    // calls Vehicle's show()
        System.out.println("Car");
    }
}
```

Q19. What is **method hiding** vs **method overriding**?

A:

- **Method overriding:** Subclass redefines an **instance method** from the parent. Resolved at **runtime (dynamic dispatch)**.
- **Method hiding:** Subclass redefines a **static method** from the parent. Resolved at **compile time** based on reference type, not object type.

```
class Parent {
    static void staticMethod() { System.out.println("Parent static"); }
    void instanceMethod() { System.out.println("Parent instance"); }
}

class Child extends Parent {
    static void staticMethod() { System.out.println("Child static"); } //
HIDING
    void instanceMethod() { System.out.println("Child instance"); }    //
OVERRIDING
}

Parent p = new Child();
p.staticMethod();    // "Parent static" – hiding, resolved by reference type
p.instanceMethod(); // "Child instance" – overriding, resolved by object type
```

Q20. Can constructors be inherited?

A: No. Constructors are not inherited because they are tied to the specific class name. However, a

subclass constructor **implicitly or explicitly calls** the parent constructor via `super ( )`.

Q21. What is the difference between **extends** and **implements**?

A:

- **extends**: Used for class-to-class inheritance (single) or interface-to-interface inheritance.
- **implements**: Used when a class adopts an interface's contract (can implement multiple interfaces).

```
class Dog extends Animal implements Trainable, Swimmable { ... }
```

5. Polymorphism

Q22. What is Polymorphism?

A: Polymorphism means "many forms." It allows **one interface to be used for different underlying data types or behaviors**. There are two main types:

- **Compile-time (Static)**: Resolved at compile time. Achieved via **method overloading** and **operator overloading**.
 - **Runtime (Dynamic)**: Resolved at runtime. Achieved via **method overriding** and **upcasting**.
-

Q23. What is compile-time polymorphism?

A: Achieved when multiple methods share the same name but differ in **parameter list (type, number, or order)**. The compiler decides which method to call.

```
class MathUtils {  
    int add(int a, int b) { return a + b; }  
    double add(double a, double b) { return a + b; }  
    int add(int a, int b, int c) { return a + b + c; }  
}
```

Return type alone does NOT differentiate overloaded methods.

Q24. What is runtime polymorphism?

A: Achieved when a **parent class reference holds a child class object**, and the overridden method is resolved based on the **actual object type at runtime** (dynamic dispatch).

```
class Shape {  
    void draw() { System.out.println("Drawing shape"); }  
}  
  
class Circle extends Shape {  
    void draw() { System.out.println("Drawing circle"); }  
}
```



```
}  
Shape s = new Circle(); // upcasting  
s.draw();               // "Drawing circle" – resolved at runtime
```

Q25. What is upcasting and downcasting?

A:

- **Upcasting:** Assigning a child object to a parent reference. Done implicitly. Safe.
- **Downcasting:** Casting a parent reference back to a child type. Must be explicit. Can throw `ClassCastException` if incorrect.

```
Animal a = new Dog(); // Upcasting (implicit)  
Dog d = (Dog) a;      // Downcasting (explicit)  
  
// Safe downcasting:  
if (a instanceof Dog) {  
    Dog d2 = (Dog) a;  
}
```

Q26. What is operator overloading?

A: Defining custom behavior for standard operators for user-defined types. Supported in C++, Python, Kotlin. **Not supported in Java** (except for + on Strings, which is built-in).

```
class Vector:  
    def __init__(self, x, y): self.x, self.y = x, y  
    def __add__(self, other): return Vector(self.x + other.x, self.y + other.y)  
  
v = Vector(1, 2) + Vector(3, 4) # calls __add__
```

6. Abstraction

Q27. What is Abstraction?

A: Abstraction means **showing only the essential details** and hiding the implementation complexity. It allows users to interact with a system without knowing how it works internally.

Real-world analogy: You use a TV remote without knowing its circuit board.

In OOP, abstraction is achieved via:

- **Abstract classes**
 - **Interfaces**
-

Q28. What is the difference between Abstraction and Encapsulation?

A:

Aspect	Abstraction	Encapsulation
Focus	What an object does	How it is protected
Goal	Reduce complexity	Increase security
Mechanism	Abstract class / Interface	Access modifiers
Design level	Design phase	Implementation phase

Abstraction is about design; encapsulation is about implementation protection.

Q29. What is an abstract class?

A: A class declared with the `abstract` keyword that **cannot be instantiated**. It may contain abstract methods (without body) and concrete methods (with body).

```
abstract class Shape {
    abstract double area();           // Must be overridden
    void display() {                 // Concrete – can be used as-is
        System.out.println("Area: " + area());
    }
}

class Circle extends Shape {
    double radius;
    Circle(double r) { this.radius = r; }
    double area() { return Math.PI * radius * radius; }
}
```

7. Constructors & Destructors

Q30. What is a constructor?

A: A special method that is automatically called when an object is created. It initializes the object's state. Key features:

- Same name as the class
 - No return type (not even `void`)
 - Can be overloaded
-

Q31. What are the types of constructors?

A:

1. **Default Constructor:** No parameters. Provided by compiler if none defined.
2. **Parameterized Constructor:** Accepts arguments to initialize fields.
3. **Copy Constructor:** Creates a new object as a copy of an existing one. (Explicit in C++,

achieved via clone/copy in Java/Python)

```
class Book {
    String title; int pages;

    Book() { title = "Unknown"; pages = 0; }           // Default
    Book(String t, int p) { title = t; pages = p; }     // Parameterized
    Book(Book other) { title = other.title; pages = other.pages; } // Copy
}
```

Q32. What is constructor chaining?

A: Calling one constructor from another within the same class (`this()`) or from a parent class (`super()`). Must be the **first statement** in the constructor.

```
class Employee {
    String name; int age; String dept;

    Employee(String name) { this(name, 25); }
    Employee(String name, int age) { this(name, age, "General"); }
    Employee(String name, int age, String dept) {
        this.name = name; this.age = age; this.dept = dept;
    }
}
```

Q33. Can a constructor be private?

A: Yes. Private constructors are used in:

- **Singleton Pattern:** Ensures only one instance is created.
- **Utility/Helper classes:** Classes with only static methods (e.g., `Math`).
- **Factory method pattern:** Control instantiation through a static method.

```
class Singleton {
    private static Singleton instance;
    private Singleton() {} // Private constructor
    public static Singleton getInstance() {
        if (instance == null) instance = new Singleton();
        return instance;
    }
}
```

Q34. What is a destructor?

A: A method called when an object is destroyed to release resources.

- **C++:** `~ClassName()` — explicit destructor.
- **Java/Python:** Garbage collector handles memory; `finalize()` in Java (deprecated) and `__del__` in Python for cleanup, but unreliable.

Best practice in Java: Use `try-with-resources` or implement `AutoCloseable` for deterministic cleanup.

Q35. What is the difference between `finalize()` and a destructor?

A:

- **Destructor (C++):** Called deterministically when object goes out of scope or `delete` is called.
- **`finalize()` (Java):** Called by the GC before collecting the object — **not deterministic**, deprecated in Java 9+.

Use `close()` / `AutoCloseable` in Java instead for resource management.

8. Interfaces & Abstract Classes

Q36. What is an Interface?

A: An interface is a **contract** that defines what a class must do (method signatures) without specifying how. A class that implements an interface must provide implementations for all its abstract methods.

```
interface Drawable {  
    void draw();  
    default void describe() {           // Default method (Java 8+)  
        System.out.println("I am drawable");  
    }  
}  
  
class Circle implements Drawable {  
    public void draw() { System.out.println("Drawing circle"); }  
}
```

Q37. What are the differences between Abstract Class and Interface?

A:

Feature	Abstract Class	Interface
Instantiation	Cannot be instantiated	Cannot be instantiated
Methods	Abstract + concrete	Abstract (+ default/static in Java 8+)
Variables	Any type	<code>public static final</code> only
Constructor	Has constructors	No constructors
Inheritance	Single	Multiple
When to use	IS-A relationship with shared state	CAN-DO / capability contract

Q38. When would you use an abstract class over an interface?

A: Use an **abstract class** when:

- Subclasses share common state (instance variables).
- You want to provide default/partial implementations.
- You have a true IS-A relationship.

Use an **interface** when:

- Multiple unrelated classes need to share a contract.
 - You need multiple inheritance.
 - You're defining a capability/role (e.g., `Serializable`, `Comparable`).
-

Q39. What are default and static methods in interfaces (Java 8+)?

A:

- **Default methods:** Provide a default implementation in the interface. Can be overridden. Added for backward compatibility.
- **Static methods:** Belong to the interface itself, not implementing classes. Called via `InterfaceName.method()`.

```
interface Logger {  
    void log(String msg);  
    default void logInfo(String msg) { log("[INFO] " + msg); } // default  
    static Logger getConsoleLogger() { return msg -> System.out.println(msg); }  
    // static  
}
```

Q40. Can an interface have a constructor?

A: No. Interfaces cannot have constructors because they cannot be instantiated. Constructors are used to initialize object state, but interfaces have no instance state (`final static` fields only).

9. Access Modifiers

Q41. What are access modifiers in OOP?

A: Access modifiers control the visibility and accessibility of class members.

Modifier	Same Class	Same Package	Subclass	Everywhere
<code>private</code>				
<code>default</code>				

Modifier	Same Class	Same Package	Subclass	Everywhere
(package)				
protected				
public				

Q42. What is the difference between **private** and **protected**?

A:

- **private:** Accessible only within the **same class**. Not even subclasses.
- **protected:** Accessible within the **same class, same package, and subclasses** (even in different packages).

```
class Animal {
    private String dna;          // Only Animal can access
    protected String species;    // Animal + subclasses can access
}

class Dog extends Animal {
    void info() {
        // System.out.println(dna);    // ERROR - private
        System.out.println(species);    // OK - protected
    }
}
```

10. Static Members

Q43. What is a static variable?

A: A variable that belongs to the **class**, not to any individual object. One shared copy exists regardless of how many objects are created. Modified with **static** keyword.

```
class Student {
    String name;
    static int totalStudents = 0; // shared

    Student(String name) {
        this.name = name;
        totalStudents++;
    }
}
```

Q44. What is a static method?

A: A method that belongs to the class itself. Can be called without creating an object. Limitations:

- Cannot access instance variables or instance methods directly.

- Cannot use `this` or `super`.

```
class MathUtils {  
    static int square(int n) { return n * n; }    // static method  
}  
MathUtils.square(5);    // No object needed
```

Q45. What is a static block?

A: A static block (static initializer) runs **once when the class is first loaded** into memory, before any object is created or static method is called. Used for complex static field initialization.

```
class Config {  
    static final String DB_URL;  
    static {  
        // Runs once at class load time  
        DB_URL = System.getenv("DB_URL") != null ? System.getenv("DB_URL") :  
"localhost";  
    }  
}
```

Q46. Can we override static methods?

A: No. Static methods are resolved at compile time using the reference type (early binding). Subclass can **hide** a static method by declaring one with the same signature, but it's not overriding. Polymorphism does not apply.

11. Method Overloading & Overriding

Q47. What is method overloading?

A: Defining **multiple methods with the same name** in the same class, differing by:

- Number of parameters
- Type of parameters
- Order of parameters

Return type alone is NOT sufficient for overloading.

```
void print(int n) { ... }  
void print(double d) { ... }  
void print(int a, int b) { ... }  
void print(String s) { ... }
```

Q48. What is method overriding?

A: A subclass **redefines a method** with the **same signature** inherited from its parent class to provide its own specific behavior.

Rules:

- Same method name, same parameters, same return type (or covariant)
- Cannot override `private`, `static`, or `final` methods
- Access modifier cannot be more restrictive in the override
- Use `@Override` annotation (Java) for compile-time verification

```
class Animal {  
    void sound() { System.out.println("Generic sound"); }  
}  
class Cat extends Animal {  
    @Override  
    void sound() { System.out.println("Meow"); }  
}
```

Q49. What is covariant return type?

A: An overriding method can return a **subtype** of the return type declared in the parent method (Java 5+).

```
class Animal {  
    Animal getInstance() { return new Animal(); }  
}  
class Dog extends Animal {  
    @Override  
    Dog getInstance() { return new Dog(); } // Covariant – Dog is a subtype of  
Animal  
}
```

Q50. Can we override a `final` method?

A: No. A `final` method cannot be overridden. It ensures the implementation remains unchanged across all subclasses. This is used when the behavior is critical and must not be altered.

12. Association, Aggregation & Composition

Q51. What is Association?

A: A relationship between two classes where one class uses or interacts with another. It represents a "uses-a" or "knows-a" relationship.

Types:

- **Unidirectional:** A knows B, but B doesn't know A.
- **Bidirectional:** Both A and B know each other.
- **Multiplicity:** One-to-one, one-to-many, many-to-many.

```
class Teacher { Student student; } // Teacher knows Student
```

Q52. What is Aggregation?

A: A "has-a" relationship where the child object **can exist independently** of the parent. It's a weak association.

```
class Department {  
    List<Employee> employees; // Employees can exist without Department  
}
```

If the Department is deleted, employees still exist.

Q53. What is Composition?

A: A "has-a" (strong) relationship where the child object **cannot exist independently** of the parent. The child's lifecycle is tied to the parent.

```
class House {  
    Room room; // Room cannot exist without House  
    House() { room = new Room(); }  
}
```

If the House is destroyed, Room is destroyed too.

Q54. Aggregation vs Composition — Key Differences?

A:

Aspect	Aggregation	Composition
Relationship	Weak HAS-A	Strong HAS-A
Independence	Part can exist independently	Part depends on whole
Lifecycle	Separate	Same
Example	Car HAS-A Driver	Car HAS-A Engine

13. Coupling & Cohesion

Q55. What is Coupling?

A: Coupling measures how **dependent classes are on each other**.

- **Tight coupling:** Classes are highly dependent. Changes in one affect the other. Bad practice.
- **Loose coupling:** Classes have minimal dependencies. Achieved using interfaces, dependency injection, and abstractions.

```
// Tight – OrderService creates its own DB connection  
class OrderService {  
    MySQLDatabase db = new MySQLDatabase(); // tightly coupled
```

```
}  
  
// Loose – depends on abstraction  
class OrderService {  
    Database db;  
    OrderService(Database db) { this.db = db; } // loosely coupled via DI  
}
```

Q56. What is Cohesion?

A: Cohesion measures how **focused and related the responsibilities** of a class are.

- **High cohesion:** A class does one thing and does it well. Preferred.
- **Low cohesion:** A class handles unrelated responsibilities. God class anti-pattern.

A class that handles user authentication, email notifications, and report generation has **low cohesion**.

14. SOLID Principles

Q57. What are the SOLID Principles?

A: SOLID is an acronym for five design principles for writing maintainable, scalable OOP code:

S — Single Responsibility Principle (SRP)

A class should have **only one reason to change**.

```
// BAD – two responsibilities  
class Report {  
    void generateReport() { ... }  
    void saveToFile() { ... }  
}  
  
// GOOD – separated  
class ReportGenerator { void generate() { ... } }  
class ReportSaver { void save(Report r) { ... } }
```

O — Open/Closed Principle (OCP)

Software entities should be **open for extension but closed for modification**.

Use inheritance or interfaces to add behavior without changing existing code.

```
interface Discount { double apply(double price); }  
class SeasonalDiscount implements Discount { ... } // extend without modifying  
base  
class LoyaltyDiscount implements Discount { ... }
```

L — Liskov Substitution Principle (LSP)

Subclass objects should be **substitutable for parent class objects** without breaking behavior.

```
// VIOLATION – Square breaking Rectangle behavior
class Rectangle { void setWidth(int w); void setHeight(int h); }
class Square extends Rectangle {
    // Square CANNOT independently set width and height – violates LSP
}
```

I — Interface Segregation Principle (ISP)

Clients should **not be forced to depend on interfaces they don't use**. Prefer smaller, specific interfaces.

```
// BAD
interface Worker { void work(); void eat(); } // Robots don't eat!

// GOOD
interface Workable { void work(); }
interface Eatable { void eat(); }
```

D — Dependency Inversion Principle (DIP)

High-level modules should **not depend on low-level modules**. Both should depend on abstractions.

```
// BAD
class LightBulb { void turnOn(); }
class Switch { LightBulb bulb; } // depends on concrete class

// GOOD
interface Switchable { void turnOn(); }
class Switch { Switchable device; } // depends on abstraction
```

15. Design Patterns

Q58. What are Design Patterns?

A: Design patterns are **reusable solutions to commonly occurring software design problems**. Categorized into three types (Gang of Four):

1. **Creational** — How objects are created.
 2. **Structural** — How classes and objects are composed.
 3. **Behavioral** — How objects communicate and interact.
-

Q59. Explain the Singleton Pattern.

A: Ensures only **one instance** of a class exists and provides a global access point.

```
class Singleton {
    private static volatile Singleton instance;
    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton(); // Double-
checked locking
            }
        }
        return instance;
    }
}
```

Use cases: Logger, DB connection pool, Config manager.

Q60. Explain the Factory Pattern.

A: Provides an interface for creating objects, letting subclasses or logic decide which class to instantiate.

```
interface Shape { void draw(); }
class Circle implements Shape { public void draw()
{ System.out.println("Circle"); } }
class Square implements Shape { public void draw()
{ System.out.println("Square"); } }

class ShapeFactory {
    static Shape create(String type) {
        return switch (type) {
            case "circle" -> new Circle();
            case "square" -> new Square();
            default -> throw new IllegalArgumentException("Unknown shape");
        };
    }
}
```

Q61. Explain the Observer Pattern.

A: Defines a one-to-many dependency: when one object changes state, all dependents are notified. Used in event systems.

```
interface Observer { void update(String event); }

class EventBus {
    List<Observer> observers = new ArrayList<>();
    void subscribe(Observer o) { observers.add(o); }
    void publish(String event) { observers.forEach(o -> o.update(event)); }
}
```

Q62. Explain the Strategy Pattern.

A: Defines a family of algorithms, encapsulates each, and makes them interchangeable at runtime.

```
interface SortStrategy { void sort(int[] arr); }
class BubbleSort implements SortStrategy { public void sort(int[] arr) { ... } }
class QuickSort implements SortStrategy { public void sort(int[] arr) { ... } }

class Sorter {
    SortStrategy strategy;
    Sorter(SortStrategy s) { this.strategy = s; }
    void sort(int[] arr) { strategy.sort(arr); }
}
```

Q63. Explain the Decorator Pattern.

A: Dynamically adds behavior to objects without modifying the class. Wraps objects in decorator classes.

```
interface Coffee { double cost(); }
class SimpleCoffee implements Coffee { public double cost() { return 1.0; } }

class MilkDecorator implements Coffee {
    Coffee coffee;
    MilkDecorator(Coffee c) { coffee = c; }
    public double cost() { return coffee.cost() + 0.5; }
}

Coffee c = new MilkDecorator(new SimpleCoffee());
// cost = 1.5
```

16. Exception Handling in OOP

Q64. How does exception handling fit into OOP?

A: Exceptions are objects — they're instances of exception classes in an inheritance hierarchy. This allows:

- **Catching by type:** Polymorphic catching of different exception types.
 - **Custom exceptions:** Extend `Exception` or `RuntimeException` for domain-specific errors.
 - **Exception chaining:** Wrap low-level exceptions in higher-level ones.
-

Q65. What is the difference between checked and unchecked exceptions?

A:

- **Checked:** Must be declared (`throws`) or handled (`try-catch`). Compile-time check. E.g., `IOException`, `SQLException`.
 - **Unchecked (Runtime):** Not required to be handled. E.g., `NullPointerException`, `ArrayIndexOutOfBoundsException`.
-

Q66. How do you create a custom exception?

A:

```
class InsufficientFundsException extends Exception {
    double amount;
    InsufficientFundsException(double amount) {
        super("Insufficient funds. Needed: " + amount);
        this.amount = amount;
    }
}

class BankAccount {
    double balance = 100;
    void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) throw new InsufficientFundsException(amount);
        balance -= amount;
    }
}
```

17. Generics & Templates

Q67. What are Generics in OOP?

A: Generics allow classes, interfaces, and methods to operate on **typed parameters**, enabling type-safe, reusable code without casting.

```
class Box<T> {
    T value;
    Box(T value) { this.value = value; }
    T get() { return value; }
}

Box<String> strBox = new Box<>("Hello");
Box<Integer> intBox = new Box<>(42);
```

Q68. What is a bounded type parameter?

A: Restricts the types that can be used as generic arguments.

```
// Only Number and its subclasses (Integer, Double, etc.)
class Stats<T extends Number> {
    T[] data;
    double sum() {
        double s = 0;
        for (T d : data) s += d.doubleValue();
        return s;
    }
}
```

18. Memory Management & Object Lifecycle

Q69. What is garbage collection?

A: Automatic memory management that reclaims memory occupied by objects no longer reachable by any reference. JVM's GC runs in the background (Mark-and-Sweep, Generational GC, etc.).

Objects eligible for GC:

- No references pointing to them.
 - References set to `null`.
 - Created inside a method (after method exits, if no reference escapes).
-

Q70. What is a memory leak in OOP languages with GC?

A: A memory leak occurs when objects are still referenced but never used again — GC cannot collect them. Common causes:

- Static collections holding objects indefinitely.
 - Event listeners not removed.
 - Unclosed streams/connections held by objects.
 - Caches with no eviction policy.
-

Q71. What is the object lifecycle?

A:

1. **Class Loading:** Class bytecode loaded into JVM.
 2. **Object Creation:** `new` allocates heap memory, constructor runs.
 3. **Usage:** Methods are called, state changes.
 4. **Unreachable:** No live references remain.
 5. **GC Collection:** Memory is reclaimed.
 6. **Finalization** (if any): `finalize()` called (deprecated in Java 9+).
-

19. Inner Classes & Anonymous Classes

Q72. What are inner classes?

A: A class defined within another class. Types:

1. **Non-static inner class:** Has access to outer class members including private.
2. **Static nested class:** Does not have access to outer instance members.
3. **Local class:** Defined inside a method.

4. **Anonymous class:** No name; instantiated and defined inline.

```
class Outer {
    class Inner { void show() { System.out.println("Inner"); } }
    static class StaticNested { void show() { System.out.println("Static Nested"); } }
}
```

Q73. What is an anonymous class?

A: A class without a name that is declared and instantiated at the same time. Often used to override methods of an interface or class inline.

```
Runnable r = new Runnable() {
    public void run() { System.out.println("Running anonymously"); }
};
r.run();
```

Largely replaced by **lambda expressions** in Java 8+.

20. Advanced OOP Concepts

Q74. What is the difference between early binding and late binding?

A:

- **Early binding (static):** Method call resolved at **compile time**. Applies to overloaded, static, and final methods. Faster.
 - **Late binding (dynamic):** Method call resolved at **runtime** based on actual object type. Applies to overridden methods. Enables polymorphism.
-

Q75. What is a virtual function/method?

A: A function defined in a base class that can be **overridden in derived classes** and is resolved via late binding (vtable in C++). In Java, all non-static, non-private, non-final methods are **implicitly virtual**.

```
class Base {
    virtual void speak() { cout << "Base" << endl; } // virtual in C++
};
class Derived : public Base {
    void speak() override { cout << "Derived" << endl; }
};
```

Q76. What is the vtable (Virtual Dispatch Table)?

A: A compiler-generated lookup table used in C++ for dynamic dispatch. Each class with virtual functions has a vtable containing pointers to its virtual method implementations. Each object stores

a hidden `vptr` pointer to its class's `vtable`.

This is how runtime polymorphism works at the machine level.

Q77. What is object cloning?

A: Creating a copy of an object.

- **Shallow copy:** Copies primitive fields and references (not the objects they point to). Both original and copy share nested objects.
- **Deep copy:** Copies all fields recursively — nested objects are also copied.

```
class Person implements Cloneable {
    String name;
    Address address;

    // Shallow
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }

    // Deep
    Person deepCopy() {
        return new Person(this.name, new Address(this.address.city));
    }
}
```

Q78. What is the Prototype Pattern?

A: A creational pattern where objects are created by cloning an existing object (prototype) instead of instantiating from scratch. Useful when object creation is expensive.

```
interface Prototype { Prototype clone(); }

class GameCharacter implements Prototype {
    String type; int health;
    public Prototype clone() { return new GameCharacter(this.type, this.health); }
}
```

Q79. What is Dependency Injection (DI)?

A: A technique where an object's dependencies are **provided externally** rather than created internally. Promotes loose coupling and testability.

Types:

- **Constructor injection** (preferred)
- **Setter injection**
- **Interface injection**

```
// Without DI – tightly coupled
class Service { Repository repo = new MySQLRepository(); }
```

```
// With DI – loosely coupled
class Service {
    Repository repo;
    Service(Repository repo) { this.repo = repo; } // Injected
}
```

Q80. What is Inversion of Control (IoC)?

A: A design principle where the control of object creation and lifecycle is **inverted** from the object itself to an external container or framework. DI is one implementation of IoC.

Frameworks like Spring (Java), Angular, and ASP.NET Core implement IoC containers.

Q81. What is the Law of Demeter (Principle of Least Knowledge)?

A: An object should only talk to its **immediate friends** and not reach through objects to access deeper neighbors.

```
// VIOLATION
double price = order.getCustomer().getWallet().getMoney();

// COMPLIANT – Customer exposes a high-level method
double price = order.getCustomerBudget();
```

Reduces coupling; changes in `Wallet` don't ripple through to order-processing code.

Q82. What is method chaining / fluent interface?

A: A pattern where methods return `this` (the current object), enabling chaining calls in a readable, expressive way.

```
class QueryBuilder {
    String table, condition;
    QueryBuilder from(String t) { this.table = t; return this; }
    QueryBuilder where(String c) { this.condition = c; return this; }
    String build() { return "SELECT * FROM " + table + " WHERE " + condition; }
}

String q = new QueryBuilder().from("users").where("age > 18").build();
```

Q83. What is a mixin?

A: A class that provides methods to be "mixed into" other classes without using traditional inheritance. Used to add reusable behaviors to multiple classes. Supported directly in Python, Dart, Kotlin (via interfaces with default methods), Ruby.

```
class JsonMixin:
    def to_json(self):
        import json
        return json.dumps(self.__dict__)

class User(JsonMixin):
```

```
def __init__(self, name, age):
    self.name, self.age = name, age

u = User("Alice", 30)
print(u.to_json())    # {"name": "Alice", "age": 30}
```

Q84. What is duck typing?

A: "If it walks like a duck and quacks like a duck, it's a duck." In duck typing (Python, JS), an object's suitability is determined by the **presence of methods/attributes**, not by the class it inherits from.

```
class Dog:
    def speak(self): return "Woof"

class Cat:
    def speak(self): return "Meow"

def make_speak(animal):
    print(animal.speak())    # Works for any object with speak()

make_speak(Dog())
make_speak(Cat())
```

Q85. What is the difference between `==` and `equals()` in Java?

A:

- `==` compares **references** (whether two variables point to the same object in memory).
- `.equals()` compares **content/value** (can be overridden to define logical equality).

```
String a = new String("hello");
String b = new String("hello");

a == b           // false – different objects
a.equals(b)      // true – same content
```

Always override `equals()` together with `hashCode()` when used in collections.

Q86. What is the `hashCode()` contract?

A:

1. If `a.equals(b)` is true, then `a.hashCode() == b.hashCode()` must be true.
2. If `a.equals(b)` is false, hashCodes *may* differ but are not required to.
3. `hashCode()` must be consistent across multiple calls for the same object.

Breaking this contract causes issues with `HashMap`, `HashSet`, etc.

Q87. What is immutability? How do you create an immutable class?

A: An immutable object's state **cannot be changed after creation**. Example: `String` in Java.

To create an immutable class:

1. Declare class as `final` (prevent subclassing).
2. Make all fields `private` and `final`.
3. No setters.
4. Initialize all fields via constructor.
5. Return deep copies for mutable fields (defensive copying).

```
final class Point {  
    private final int x, y;  
    Point(int x, int y) { this.x = x; this.y = y; }  
    int getX() { return x; }  
    int getY() { return y; }  
}
```

Benefits: Thread-safe, safe to share, easier to reason about.

Q88. What is the difference between `final`, `finally`, and `finalize`?

A:

- `final`: Keyword. Variable = constant, method = cannot override, class = cannot extend.
 - `finally`: Block in try-catch that **always executes** (cleanup code).
 - `finalize()`: Method called by GC before object destruction (deprecated in Java 9+).
-

Q89. What is a functional interface?

A: An interface with exactly **one abstract method**, enabling it to be used as a lambda expression target. (Java 8+)

```
@FunctionalInterface  
interface Transformer {  
    int transform(int n);  
}  
  
Transformer doubler = n -> n * 2;  
System.out.println(doubler.transform(5)); // 10
```

Built-in: `Runnable`, `Callable`, `Comparator`, `Predicate<T>`, `Function<T, R>`.

Q90. What is method reference in Java?

A: A shorthand for lambdas that call an existing method. Types:

- `ClassName::staticMethod`
- `instance::instanceMethod`

- `ClassName::instanceMethod`
- `ClassName::new` (constructor reference)

```
List<String> names = List.of("Alice", "Bob");  
names.forEach(System.out::println); // Method reference
```

Q91. What is the difference between composition and inheritance for code reuse?

A:

"Favor composition over inheritance" — GoF

Aspect	Inheritance	Composition
Relationship	IS-A	HAS-A
Coupling	Tight	Loose
Flexibility	Limited	High
Reuse	Vertical (hierarchy)	Horizontal (plug-in)
Change impact	Fragile base class problem	Isolated changes

Inheritance is appropriate for true IS-A; composition is better when you want to reuse behavior without a strict hierarchy.

Q92. What is the Fragile Base Class Problem?

A: Changes to a base class can **unexpectedly break subclasses** because subclasses depend on the internal behavior of the base class — even for methods they don't override.

Solution: Prefer composition; use `final` to prevent subclassing where stability is critical; document the class's contract for inheritance.

Q93. What is Object Serialization?

A: The process of converting an object's state into a byte stream for storage or transmission. Deserialization reconstructs the object.

```
class User implements Serializable {  
    String name;  
    transient String password; // transient fields are NOT serialized  
}
```

`transient` keyword excludes sensitive or non-serializable fields.

Q94. What is the difference between deep and shallow copy in context of serialization?

A: Serialization can be used as a **deep copy trick** — serialize an object to a byte stream and deserialize it back to get a completely independent copy of the entire object graph (deep copy).

Q95. What is reflection in OOP?

A: Reflection allows a program to **inspect and manipulate its own class structure at runtime** — accessing fields, methods, constructors, and annotations dynamically.

```
Class<?> cls = Class.forName("com.example.Dog");
Method m = cls.getMethod("bark");
Object dog = cls.newInstance();
m.invoke(dog);
```

Used in: frameworks, DI containers, serialization, testing, ORM.

Q96. What is cohesion vs coupling summary?

A:

High cohesion + Low coupling = Good OOP Design

- **High cohesion:** Each class is focused on a single, well-defined purpose.
 - **Low coupling:** Classes depend on abstractions, not concrete implementations.
-

Q97. What are GRASP principles?

A: General Responsibility Assignment Software Patterns — guidelines for assigning responsibilities to classes:

- **Information Expert:** Assign responsibility to the class that has the data.
 - **Creator:** Assign object creation responsibility to those who aggregate or contain the object.
 - **Controller:** Assign UI event handling to a controller class.
 - **Low Coupling / High Cohesion:** Design goal.
 - **Polymorphism:** Use polymorphism to handle variation.
 - **Indirection:** Introduce a mediator to reduce coupling.
 - **Protected Variations:** Identify likely change points and wrap them behind interfaces.
-

Q98. What is the difference between object-oriented and object-based programming?

A:

- **Object-based:** Supports objects and encapsulation but **not inheritance or polymorphism**. Example: JavaScript (pre-ES6 classes), VBScript.
 - **Object-oriented:** Full OOP support — encapsulation, inheritance, polymorphism, and abstraction. Example: Java, C++, Python.
-

Q99. What is a God Class? Why is it bad?

A: A God Class is an anti-pattern where **one class knows too much or does too much** — it handles many unrelated responsibilities. Problems:

- Low cohesion
- Hard to test, maintain, and extend
- Tightly couples unrelated subsystems

Solution: Decompose into smaller, focused classes following SRP.

Q100. What is the difference between **abstract class** and **concrete class**?

A:

- **Abstract class:** Cannot be instantiated. May have abstract methods that must be implemented by subclasses. Serves as a partial or full blueprint.
 - **Concrete class:** Fully implemented class that can be instantiated. All methods have bodies.
-

☐ Quick Reference Cheat Sheet

Concept	One-liner
Encapsulation	Bundle data + methods; hide internals
Inheritance	Child acquires parent's properties
Polymorphism	One interface, many implementations
Abstraction	Show what, hide how
Association	"uses-a" relationship
Aggregation	"has-a" (weak, independent lifecycle)
Composition	"has-a" (strong, dependent lifecycle)
Coupling	How dependent classes are
Cohesion	How focused a class is
SRP	One class, one reason to change
OCP	Open to extend, closed to modify
LSP	Subtypes must be substitutable
ISP	Prefer small, specific interfaces
DIP	Depend on abstractions
Singleton	One instance globally
Factory	Delegate object creation
Observer	Notify dependents on state change
Strategy	Swap algorithms at runtime
Decorator	Add behavior by wrapping

Good luck with your interviews!